Ask, Don't Guess: Targeted Clarification for Speech Recognition Errors

Sankalp Parashar **Preethi Jyothi** **Sunita Sarawagi**
IIT Bombay
sankalp.parashar.c2024@iitbombay.org

arXiv:2610.24809v1 [cs.CL] 17 Oct 2026

Abstract

Speech recognition in real time in far-field conversational settings still incurs significant errors. Identifying potential error spans during streaming ASR permits a recourse mechanism where the speaker can be asked to repeat the affected content under cleaner acoustic conditions. We propose PARDON, a new framework for online error localization, that identifies spans likely to benefit from this intervention. Our supervision uses clean audio to estimate how the recognizer's output might change under improved acoustic conditions. We probabilistically align noisy and clean decodes using a pair-HMM to obtain soft supervision. These soft targets train a counterfactual gain scorer by combining a suite of acoustic and language features to produce token-level scores, that are further grouped into spans for targeted recourse. An LLM uses the ASR transcript of the repetition to revise only the selected span. At a precision of 70%, PARDON achieves the highest recall by identifying up to 52% more erroneous tokens compared to the strongest baseline. Targeted repetition yields relative WER reductions of up to 24% under a budget that accounts for both interruption frequency and the lengths of re-uttered spans.

1 Introduction

Streaming ASR predicts transcriptions in real time as the model processes incoming speech. However, ambient noise or interfering speakers could obscure parts of the speech signal, leading to localized perception errors. Retrospectively identifying these potential erroneous spans in an evolving transcript is our primary problem of interest. Addressing this problem also has practical value because the online setting naturally supports a recourse mechanism. Once the error spans are localized, a timely request to the speaker to repeat the affected segment under better conditions could improve transcription accuracy.

Error localization has been previously studied in the context of spoken dialogue systems (Skantze, 2007) and through confidence estimation modules for streaming ASR (Ragni et al., 2018; Qiu et al., 2021). Supervision in these prior works typically comes from reference transcripts. However, reference-based supervision targets transcript correctness and not the potential benefit of re-utterance: some errors, such as those arising out of homophones, persist even after re-utterance under clean acoustic condition. This motivates using targets that directly reflect how the recognizer's output might change under cleaner acoustic conditions.

In order to support targeted recourse, we propose a new framework for online error localization in streaming ASR called PARDON, short for **P**redicting **A**coustic **R**e-utterance **D**esirability **O**ver **N**oisy spans. PARDON derives supervision from a *counterfactual estimate* of what the recognizer would output if the same content were heard under cleaner acoustic conditions. We retrospectively compare these clean decodes with the original noisy decodes by marginalizing over multiple plausible alignments of the clean and noisy decodes using a pair hidden Markov model (pair-HMM). The pair-HMM assigns each token a soft label given by the posterior probability of a recognition error at that position. These soft labels supervise a counterfactual gain estimator that takes as input both language features and acoustic features extracted via a streaming posterior alignment with audio frames. This estimator runs alongside the ASR's decodes producing token-level scores, which are grouped to identify spans that could benefit from re-utterance.

For recourse, the speaker is asked to repeat the identified span under cleaner acoustic conditions.[1] The ASR model transcribes the repetition and an

[1]We use targeted repetition as an initial recourse strategy. We leave the design of suitable user interfaces, recourse requests beyond repetition and improved policies for user interruptions to future work.

LLM uses the resulting transcript to revise only the selected span, after which decoding continues.

Our contributions can be summarized as follows: (1) We introduce PARDON, a new framework for online error localization in streaming ASR that isolates potentially erroneous spans through counterfactual supervision from clean audio and a pair-HMM that accounts for uncertainty in the error locations. To the best of our knowledge, the online error localization setting we consider is itself new. (2) We develop a streaming counterfactual gain scorer that combines acoustic and language features, with supervision from the pair-HMM, to identify spans for targeted recourse through repetition. (3) For error localization, we obtain the highest average precision across four challenging corpora of conversational speech, with the largest margins in the high-precision regime, where we recall up to 52% more of the erroneous tokens than the strongest baseline at a precision of 70%. (4) We harness these localized errors for targeted recourse and reduce the word error rate by up to 24% relative, and remove up to 30% more of the errors than recent prior work under the same budget on the number of speaker interruptions and re-uttered words.

2 Related Work

Localizing recognition errors. Word-level confidence estimation has a long history in speech recognition (Jiang, 2005). For end-to-end models, confidence is obtained either directly from the output distribution, where entropy-based measures detect incorrect words in CTC and RNN-T models considerably better than the token probability (Laptev and Ginsburg, 2022), or from a learned estimator over the decoder's internal states (Qiu et al., 2021). Most of this work aims to detect recognition errors in general. We instead estimate the gain from re-uttering the audio under cleaner conditions. This shifts our solution in four ways: how the detector is supervised, which acoustic features it uses, how evidence is pooled over the transducer's alignment lattice, and how it runs in a streaming setup alongside standard ASR decoding, where flagged token-level scores are converted to spans for online recourse.

Post-hoc correction. Recognition output can also be corrected after the fact, by sequence-to-sequence correctors (Leng et al., 2021) and, more recently, by large language models that rewrite the transcript from the recognizer's N-best hypotheses (Ma et al., 2023; Yang et al., 2023; Chen et al., 2023), optionally conditioned on the noise (Hu et al., 2024). Such correction is complementary to clarification: it draws on linguistic context rather than new acoustic evidence, and could be applied on top of a clarified transcript. We include a few-shot prompted N-best corrector as a zero-interruption reference point.

Humans in the loop. Prior work on human-assisted ASR correction has largely focused on spoken dialogue systems (Bohus and Rudnicky, 2005; Skantze, 2007). These systems serve a single task, such as booking a room or navigation, and treat anything the user says unrelated to the task as an error. Furthermore, these systems often ask the user to repeat or rephrase the entire turn rather than localizing the error. More recent work involves users in correcting open-ended speech: He et al. (2025) let meeting participants correct speaker attributions with brief feedback, and Peng et al. (2026) diagnose the causes of recognition errors and resolve them through multi-turn clarification. Unlike Peng et al. (2026), who seek clarification after each turn, PARDON runs synchronously with the streaming decoder and interrupts the speaker when it identifies a span where repetition is likely to help.

3 Our Approach

Figure 1 shows the pipeline of PARDON. Let $\hat{X}$ denote an audio captured under noisy acoustic conditions, that is fed to a frozen streaming RNN-transducer $\mathcal{M}_\theta$, that produces a decode $\hat{Y}$ as explained in Section 3.1. A centerpiece of PARDON is a CF-Gain Scorer that after each decoded token $\hat{y}_j$, synchronously emits a counterfactual gain probability e_j that a re-utterance of the corresponding audio would improve the transcript, together with two deletion estimates d_j and ℓ_j (§3.2). These scores are used by the recourse module to decide when to ask for a re-utterance; in our initial, simple interface, it pools them into spans as the transcript grows, and as soon as a noise-corrupted span is identified, asks the speaker to repeat the words in the selected span (§3.3.1). The repetition is decoded by the same transducer $\mathcal{M}_\theta$, and the resulting clarification transcript is passed, together with the noisy transcript $\hat{Y}$, to an LLM corrector that produces the corrected transcript $\hat{Y}'$, after which decoding continues (§3.3).

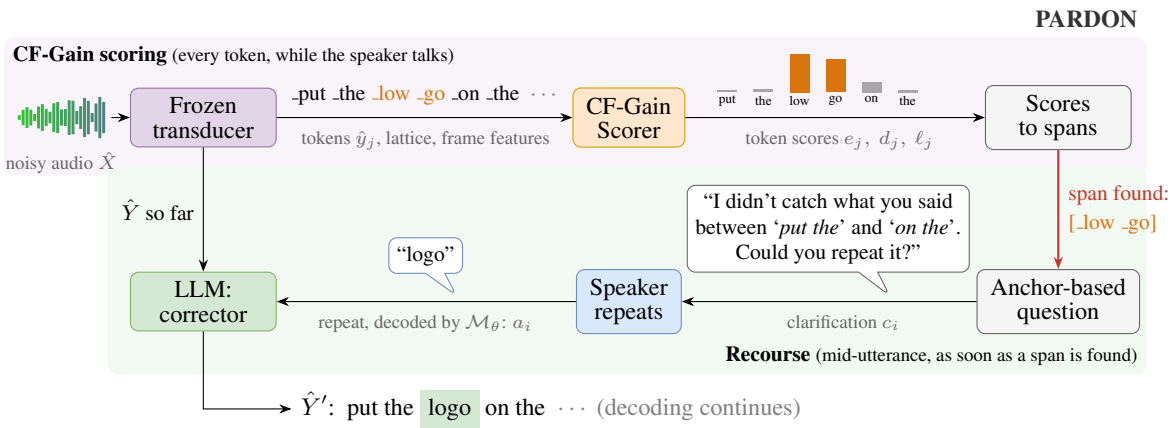


Figure 1: PARDON overview. As the speaker talks, the CF-Gain Scorer assigns every emitted token a score e_j and two deletion estimates d_j and ℓ_j, and the scores are grouped into spans. Once a span closes, PARDON asks the speaker to repeat it, using an anchor-based question. An LLM corrector puts the repeated words into the transcript.

3.1 Background of RNN-T

An RNN-T has three components. The encoder maps $\hat{X}$ to states $h[1], \ldots, h[T]$, one per frame; in a streaming model, $h[t]$ sees no audio beyond frame t apart from a fixed look-ahead. The predictor encodes the tokens emitted so far using an RNN : $g[j]$ summarizes $\hat{y}_1, \ldots, \hat{y}_{j-1}$. The joint network combines the two into a distribution over the vocabulary $\mathcal{V}$ and the blank ϵ:

$$P(\cdot \mid t, j) = \text{softmax}\big(\text{Joint}(h[t], g[j])\big). \quad (1)$$

Greedy decoding scans the frames, emitting the most probable symbol at each step: it stays at frame t after a token and moves to $t{+}1$ after a blank. We write t_j for the frame at which $\hat{y}_j$ is emitted.

3.2 The CF-Gain Scorer

After each predicted token from the noisy audio, the CF-Gain scorer estimates the counterfactual probability that a re-utterance of the corresponding audio would lead to a gain in transcription quality. More precisely, we model $p_\theta(E_j = 1 | \hat{X}, \hat{y}_1, \ldots \hat{y}_j)$ where E_j denotes an error between $j-1$ and j that can be eliminated with a clean re-utterance of the audio after the previous token $\hat{y}_{j-1}$. The error E_j could be either because of missing some token between $\hat{y}_{j-1}$ or $\hat{y}_j$, or because the token $\hat{y}_j$ is wrong. Furthermore, to estimate the amount of error that a re-utterance can recover, we also estimate the number of deletions in the gap between $\hat{y}_{j-1}$ and $\hat{y}_j$, through d_j, the probability that an error at $\hat{y}_j$ is a deletion, and ℓ_j, the number of deleted words in the gap.

To supervise the training of this estimator, we assume a training dataset $\mathcal{D} = \{\hat{X}_i, Y_i, X_i^c : i = 1 \ldots N\}$ denoting the same reference text Y_i uttered under noisy settings $\hat{X}_i$ and clean settings X_i^c. Such datasets are increasingly available in the public domain, typically obtained by recording conversations with far-field and close-talking microphones, and we experiment with four such in Section 5. If clean audio is not available, we show that even synthetically generated TTS audio suffices as distant supervision for our CF-gain scorer. We present the design of the CF-scorer in Section 3.2.1 and describe its training in Section 3.2.2.

3.2.1 Design of the CF-Gain scorer

We implement the CF-Gain scorer $p_\theta(E_j = 1 | \hat{X}, \hat{y}_1, \ldots \hat{y}_j) := e_j$ as another LSTM, much like the LM predictor, with two additional output heads for d_j and ℓ_j (Figure 2). The input to the LSTM is a set of features $\mathbf{f}_j \in \mathbf{R}^d$ extracted from $\hat{X}, \hat{y}_1, \ldots \hat{y}_j$. Instead of processing them afresh, we extract the feature set $\mathbf{f}_j$ by tapping the current state of the RNN-T. The features in $\mathbf{f}_j$ capture evidence from the audio frames, the LM state, and the joint token probabilities.

Features from audio. At each frame t, the encoder reads the audio frames up to t and outputs a hidden state $h^{(l)}[t]$ at each layer l. We extract the outputs of the final encoder layer and of one intermediate layer. In addition we extract a voice-activity probability $v[t]$ that frame t contains speech. We pool these per frame to tokens through the alignment lattice of the RNN-T.

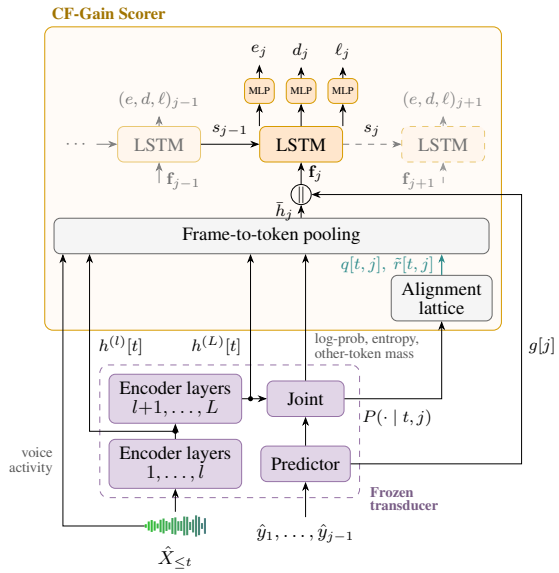


Figure 2: The CF-Gain Scorer (shaded) and the frozen transducer it reads (dashed). The emission and dwell posteriors q and $\tilde{r}$ of the streaming alignment lattice pool frame-level evidence to each token. The pooled vector $\bar{h}_j$ is concatenated with the predictor state $g[j]$ to form $\mathbf{f}_j$. A unidirectional LSTM then scores each token as it is emitted via three heads: giving fixable error score e_j and the deletion estimates d_j and ℓ_j (§3.2.2).

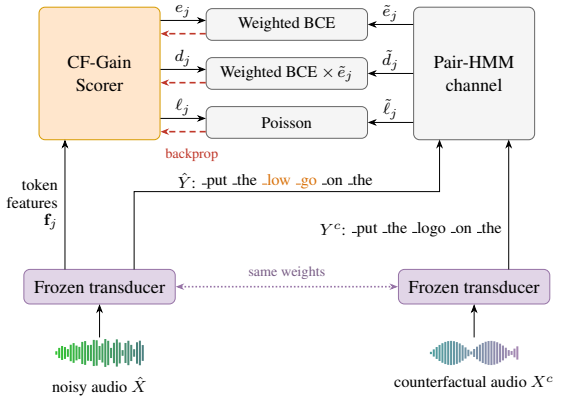


Figure 3: Training supervision. The same frozen transducer decodes the noisy audio $\hat{X}$ and the counterfactual audio X^c. A pair-HMM channel aligns the noisy decode $\hat{Y}$ to Y^c and gives, for each token, the targets $\tilde{e}_j$, $\tilde{d}_j$ and $\tilde{\ell}_j$ of the scorer's three heads; only the scorer receives gradients (§3.2.2).

Alignment lattice. When decoding $\hat{y}_j$ at frame t_j, we create a lattice over all possible alignments of the tokens $\hat{y}_{j'}$, $j' \le j$, to the frames $t \le t_j$, by restricting the joint distribution $P(\cdot \mid t, j')$ (Eq. (1)) to $\hat{y}_{j'}$ and ϵ. Furthermore, we limit the lattice to a band around the greedy emission path, which limits the alignments to those around the greedy decode (Appendix A). Forward–backward over this lattice gives the emission posterior $q[t,j]$, that $\hat{y}_j$ is emitted at frame t, and the dwell posterior $r[t,j]$, that the decoder advances past frame t while waiting to emit $\hat{y}_j$.

We pool the frame-level audio features to each token under two weightings. The emission posterior $q[\cdot, j]$ concentrates on the frames at which $\hat{y}_j$ was emitted. The normalized dwell posterior $\tilde{r}[t,j] = r[t,j] / \sum_{t'} r[t',j]$ covers the frames the decoder passed over while waiting to emit $\hat{y}_j$. For layer l, the emission view $\bar{h}_j^{\text{emit},(l)} = \sum_t q[t,j]\, h^{(l)}[t]$ describes the audio of the word that was output, and the dwell view $\bar{h}_j^{\text{dwell},(l)}$, the same sum weighted by $\tilde{r}$, describes the audio the decoder skipped, which is where a deleted word would have been. We similarly pool the voice activity score $v[t]$.

Features from the joint network. We compute a set of scalars from the distribution $P(\cdot \mid t, j)$ output by the joint network such as its entropy, the log-probability of the emitted token $\hat{y}_j$, and the other-token mass, the probability of emitting a token that is neither $\hat{y}_j$ nor the blank. These scalars are defined at every lattice node (t, j), so we pool them to tokens with q and $\tilde{r}$ as well. Pooling them in different ways gives 20 scalars, listed in Appendix B.

Features from LM. From the text branch of the transducer, we include $g[j]$ as a feature that summarizes $\hat{y}_1, \ldots, \hat{y}_{j-1}$. This feature is tied to the token, and does not require pooling.

3.2.2 Training the CF-Gain scorer

Figure 3 gives an overview of our method of training the gain scorer given dataset $\mathcal{D} = \{\hat{X}_i, Y_i, X_i^c : i = 1 \ldots N\}$. For each instance i, we obtain the decode $\hat{Y}_i$ from the noisy audio $\hat{X}_i$, and the decode $Y_i^c = (y_1^c, \ldots, y_I^c)$ of the counterfactual clean rendering X_i^c, by the same frozen recognizer $\mathcal{M}_\theta$. Comparing these sequence-level decodes, we need to get token-level supervision at each j. Since the sequences, can be differing in multiple positions, we marginalize over all possible alignments of the two sequences as follows.

For a fixed alignment A between $\hat{Y}_i$ and Y_i^c, each noisy token gets three labels: (1) $E_j(A) \in \{0, 1\}$ that denotes if as per this alignment the j-th token in $\hat{Y}_i$ is inserted, substituted, or a token before it is deleted, (2) $D_j(A) \in \{0, 1\}$ set to 1 if words are deleted before $\hat{y}_j$, and (3) $L_j(A)$ representing the number of deleted words. Words deleted after the

last token have no following token, so we append an end-of-sequence position $\hat{y}_{N+1}$ that takes their label. We marginalize these labels over all possible alignments efficiently using a pair hidden Markov model (Durbin et al., 1998).

A pair-HMM takes as input the two sequences $\hat{Y}_i$ and Y_i^c and gives the posterior probability $P(A \mid Y_i^c, \hat{Y}_i)$ of each alignment. Using an efficient forward–backward algorithm over the pair HMM, we can calculate the marginal probability of the error at $\hat{y}_j$ as

$$\tilde{e}_{ij} = \textstyle\sum_A P(A \mid Y_i^c, \hat{Y}_i)\, E_j(A). \tag{2}$$

We marginalize $D_j(A)$ over the alignments in the same way and divide it by $\tilde{e}_{ij}$, which gives $\tilde{d}_{ij}$, the probability that the error at $\hat{y}_j$ is a deletion. The pair-HMM requires a set of hyper-parameters, and we fit these via a standard expectation maximization procedure as described in Appendix H.

Training Objective. To train the scorer, we use weighted binary cross entropy (BCE) losses for the two probabilities and a Poisson loss for the deletion count as follows:

$$\min_\theta \textstyle\sum_{i,j} \Big[\mathcal{B}_w(\tilde{e}_{ij}, e_{ij}) + \tilde{e}_{ij}\, \mathcal{B}_{w_d}(\tilde{d}_{ij}, d_{ij}) + \mathbf{1}[\tilde{\ell}_{ij} \geq 1]\, \mathcal{P}(\tilde{\ell}_{ij}, \ell_{ij}) \Big] \tag{3}$$

Here $\mathcal{B}_w(y, p)$ is the weighted BCE loss of the prediction p against the target y, and $\mathcal{P}(\tilde{\ell}, \ell)$ is the Poisson loss of $\tilde{\ell} - 1$ with mean $\ell - 1$. The BCE weights w and w_d are used to upweight the positive term in each BCE loss, respectively, and are estimated such that the balance of positive and negative supervision is equal. The deletion loss is weighted by $\tilde{e}_{ij}$, so that the deletion head focuses on distinguishing deletions from other error types.

3.3 Recourse

3.3.1 From tokens to spans

Noise-induced damage tends to extend over several consecutive tokens. It is also more natural to ask a speaker to repeat a stretch of words than to ask several questions about separate words. We therefore group the token scores into spans of words as the transcript is produced. Each token is first assigned a score m_j — the expected number of errors that a re-utterance would correct at $\hat{y}_j$ and in the gap before it:

$$m_j = e_j \left[(1 - d_j) + d_j\, \ell_j \right]. \tag{4}$$

An error at $\hat{y}_j$ is either a substitution or insertion with probability $1 - d_j$, that costs one word, or a deletion with probability d_j, that costs ℓ_j words. Each word k gets the highest m_j among its tokens, written as $\bar{m}_k$.

We assume that each interruption costs c_I and each re-uttered word costs c_R, where each word is re-uttered together with content in the gap before it. A word that is not selected for repetition costs $\bar{m}_k$, i.e., the expected number of errors left uncorrected at that word and in the preceding gap. We can use a cumulative sum (CUSUM; Page, 1954) algorithm to compute the best spans in an online setting. More details of the method in Appendix C.1.

3.3.2 Clarification and correction

The request to repeat a span is a fixed template that quotes the two words on either side of the span, its anchors, and asks the speaker to restate what they said in between, e.g. "I didn't catch what you said between 'L' and 'R'. Could you repeat it?". Spans at the start or end of the utterance have only one anchor and use a shorter template (Appendix E).

For each span, the LLM corrector receives the transcript decoded up to the interruption, with the span marked and earlier spans already corrected, together with the request and the clarification transcript. It returns a replacement for the words of the span, and the corrected transcript $\hat{Y}'$ leaves every other word unchanged, so the corrector can only alter the stretches the speaker was asked about.

4 Experimental Setup

ASR System. All experiments use Nemotron Speech Streaming (NVIDIA, 2026), a 0.6B-parameter English transducer with a FastConformer (Rekesh et al., 2023) encoder of 24 layers, a 1024-dimensional encoder output at a frame rate of 80 ms, and a vocabulary of 1024 subword tokens plus the blank. We use its default attention context of 70 frames to the left and 13 to the right, so the encoder consumes audio in chunks of 1.12 s with at most 1.04 s of look-ahead. The transducer is frozen throughout.

Datasets. We use four corpora of conversational speech: two of them contain meeting conversations, AMI (Carletta et al., 2006) and NOTSOFAR-1 (Vinnikov et al., 2024), and two of them contain dinner party conversations, CHiME-6 (Watanabe et al., 2020) and DiPCo (Van Segbroeck et al., 2020). All four corpora contain both far-field

Scorer	AMI			NOTSOFAR-1			CHiME-6			DiPCo		
	AP	Recall at prec.		AP	Recall at prec.		AP	Recall at prec.		AP	Recall at prec.	
		50	70		50	70		50	70		50	70
Entropy	38.7	23.8	4.0	39.2	28.6	6.3	51.0	49.7	18.5	41.3	29.0	**11.0**
Qiu et al.	55.1	58.5	25.3	56.5	61.6	25.9	65.7	80.0	44.4	42.1	33.0	2.5
Wang et al.	52.9	56.3	18.3	50.5	51.1	13.7	62.4	75.5	36.1	35.4	9.2	0.0
TruCLeS	47.7	41.9	13.1	45.5	35.2	8.8	58.6	67.2	30.0	32.3	1.9	–
Peng et al.	47.2	46.0	3.8	49.5	54.5	7.8	57.1	72.6	11.2	41.1	25.9	1.7
CF-Gain	**56.5**	**61.7**	**32.5**	**59.8**	**68.7**	**39.4**	**66.7**	**80.3**	**49.4**	**44.1**	**36.6**	10.0

Table 1: Token-level scoring on the eval set of each corpus, against labels from the close-talking decode. Trained scorers are trained on the same corpus; the CF-Gain Scorer is evaluated by e_j. Share of erroneous tokens (the AP of a random scorer): AMI 21.5, NOTSOFAR-1 16.4, CHiME-6 24.8, DiPCo 19.3. Best result for each metric within each corpus is in bold; – indicates that the target precision was never reached.

channels and a close-talking recording of every speaker (a headset in AMI, a close-talking microphone in NOTSOFAR-1 and DiPCo, and a binaural microphone in CHiME-6). We use the official session-level train/development/evaluation splits (AMI Consortium, 2005; Cornell et al., 2024). Each utterance is a speaker turn extracted from both far-field and close-talking recordings using reference timestamps. Very short turns and turns containing reference words marked as unintelligible are excluded from training; development and evaluation sets retain all turns. The development (dev) set is used for model selection and calibration, and all numbers are reported on the evaluation (eval) set.

To show how PARDON performs in the absence of a parallel close-talking recording, we also create a synthetic rendering of each training utterance by synthesizing its reference transcript with a text-to-speech (TTS) model, Kokoro-82M (hexgrad, 2025). Dev and eval labels come from the close-talking recordings. Appendix D describes the corpora, how the turns are cut, speech synthesis details and the train/dev/eval set sizes after segmentation.

4.1 Implementation Details

We derive supervision by aligning normalized noisy and clean decodes by the same ASR system using a pair-HMM, and copying word-level labels to its subwords. Token features combine emission- and dwell-pooled encoder states from 13th and 24th blocks with the predictor state and scalar features of §3.2.1; voice activity probabilities come from Silero VAD (Silero Team, 2021). The CF-Gain Scorer uses a single-layer unidirectional LSTM with 128 hidden units and 3 prediction heads of §3.2.2. Remaining implementation details are in Appendix N.

4.2 Baselines

Token-level scoring. We compare the CF-Gain Scorer with five baseline scorers: an entropy-based method and four learned confidence estimators. i) Laptev and Ginsburg (2022): Tsallis entropy of the recognizer's output distribution at token emission, with α tuned on the development set. ii) Qiu et al. (2021): a Transformer over hypothesis tokens that attends to encoder frames and predicts token errors and deletions. iii) Wang et al. (2021): a Transformer over token embeddings and encoder frames at emission. iv) TruCLeS (Ravi et al., 2025): an LSTM predicting confidence based on lexical similarity to the reference. v) Peng et al. (2026): detects errors from the recognizer's internal states with separate perception-error and deletion detectors, whose scores we combine. We omit their comprehension-error detector from our experiments as it targets errors not repairable by clean re-utterance. Every estimator is trained on re-utterance gains from close-talking decodes rather than reference-based labels, on the same data as the CF-Gain Scorer, and adapted to streaming, using only input available at token emission (Appendix G).

Recourse. We compare PARDON with two baselines: one without user clarification and the other using it exactly the same way as PARDON. i) N-best correction: Following HyPoradise (Chen et al., 2023), an LLM corrects the transcript from a 5-best list: the greedy transcript and the four best distinct hypotheses from a width-32 beam search. We use its hypotheses-to-transcription (H2T) prompt with Gemini 3.8 Flash and 10 examples from the dev set (Appendix F.2). We also report the oracle that picks the best hypothesis from the list. ii) Proactive ASR (Peng et al., 2026): We use the same recourse pipeline as our system, with

Test	Scorer	Trained on	AP	Recall at prec. 50	70
NOTSOFAR-1	Entropy	–	39.2	28.6	6.3
	CF-Gain	NOTSOFAR-1	59.8	68.7	39.4
	CF-Gain	AMI	**63.2**	**71.7**	**39.7**
DiPCo	Entropy	–	41.3	29.0	11.0
	CF-Gain	DiPCo	44.1	36.6	10.0
	CF-Gain	CHiME-6	**54.8**	**58.0**	**24.4**

Table 2: A CF-Gain Scorer trained on one corpus is applied unchanged to the other corpus of the same genre, and compared with a scorer trained on the target corpus along with the training-free entropy baseline. Best result per block in bold.

Corpus	Supervision	AP	Recall at prec. 50	70
AMI	Close-talk	**56.5**	**61.7**	**32.5**
	TTS	54.7	59.1	27.4
NOTSOFAR-1	Close-talk	**59.8**	**68.7**	**39.4**
	TTS	51.5	54.8	15.9
CHiME-6	Close-talk	**66.7**	80.3	**49.4**
	TTS	65.3	**81.0**	44.7
DiPCo	Close-talk	**44.1**	**36.6**	**10.0**
	TTS	43.7	34.1	8.2

Table 3: CF-Gain Scorers trained with labels from the close-talking recording or from a synthesized (TTS) rendering of the reference, both evaluated against close-talk labels on the evaluation set. Best result per corpus in bold.

Peng et al. (2026) used for localizing the errors.

4.3 Evaluation metrics

Token-level scoring. Token scores e_j are evaluated on the eval set of each corpus against the hard labels E_j from the close-talking decode, using average precision (AP) and recall at precisions of 0.5 and 0.7. We also report the share of erroneous tokens, which is the AP of a random scorer.

Recourse. Recourse is evaluated by the relative WER reduction over the far-field greedy transcript against gold references, under a *re-utterance burden* $B = n_I + \beta\, n_W$. Here, n_I and n_W count interruptions and re-uttered words per minute of audio, respectively; we use $\beta \in \{0.05, 0.10\}$. The speaker's reply is simulated by the close-talking recording of the span or by TTS of its reference words. Further recourse details are in Appendix E, some recourse examples are in Appendix I, and tuning details for costs (c_R, c_I) on dev set are in Appendix M.

5 Results and Analysis

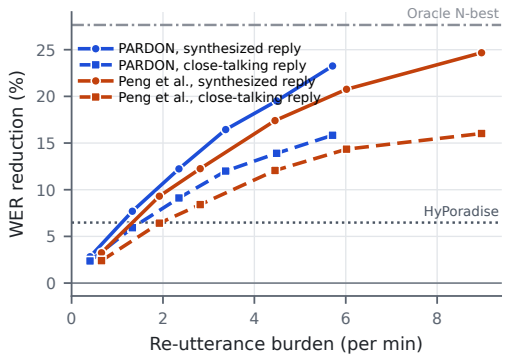
5.1 Token-level scoring

Scoring accuracy. Table 1 compares the CF-Gain Scorer with the five scoring baselines on each corpus. The CF-Gain Scorer has the highest AP on all four corpora scoring *1.0 to 3.3 points* above the strongest baseline, Qiu et al. (2021). The margin is *larger at high precision*: at a precision of 70%, it recalls 7.2, 13.5 and 5.0 points more of the erroneous tokens than the best baseline on AMI, NOTSOFAR-1 and CHiME-6.

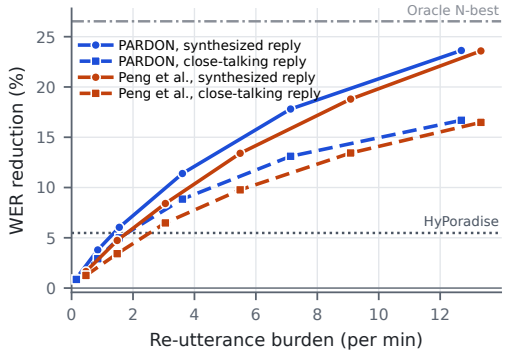
Out-of-domain generalization. To check how well the CF-Gain Scorer performs out of domain, we apply a scorer trained on one corpus unchanged to the other corpus of the same type: from AMI to NOTSOFAR-1 for meetings, and from CHiME-6 to DiPCo for dinner parties (Table 2). On NOTSOFAR-1, the scorer trained on AMI exceeds the in-domain scorer on every metric by 3.4 AP points. On DiPCo, the scorer trained on CHiME-6 exceeds the in-domain scorer on every metric by 10.7 AP points. In both cases, the source corpus has far more training data than the target (62.9 vs. 6.2 hours and 28.5 vs. 0.7 hours; Table 5), which suggests that a scorer trained on a larger corpus of the same genre can replace one trained in-domain. With a subset of the source as large as the target's training set, the transferred scorer still stays within 4 AP points of the in-domain one (Appendix J).

Synthetic supervision. We also train a scorer on each corpus with labels from TTS of the reference (Appendix D.4) instead of the close-talking recording, and evaluate it against the close-talk labels (Table 3). On AMI, CHiME-6 and DiPCo, TTS loses at most 1.8 points of AP and 5.1 points of recall at 70% precision. On AMI and CHiME-6 it still recalls more at 70% precision than every baseline of Table 1, all trained with close-talk labels, and on DiPCo it still has a higher AP than all of them. NOTSOFAR-1 is the exception: there it loses 8.3 points of AP and 23.5 points of recall at 70% precision, and falls below Qiu et al. (2021).

Ablations. Table 4 removes one group of features at a time from the CF-Gain Scorer, on AMI and CHiME-6. The intermediate encoder layer and the joint-network scalars both matter. Without layer 13, recall at 70% precision drops by 8.4 points on AMI and by 3.1 points on CHiME-6. Without the joint-network scalars, it drops by 4.3 and 8.4



(a) AMI



(b) CHiME-6

Figure 4: Recourse on the AMI (a) and CHiME-6 (b) eval sets ($\beta = 0.05$, spans of at most six words): relative WER reduction against the re-utterance burden with synthesized (solid) or close-talking (dashed) replies. PARDON uses the CF-Gain Scorer; Peng et al. uses the same recourse pipeline with the detectors of Peng et al. (2026). Horizontal lines: the zero-interruption N-best correction of HyPoradise and the N-best oracle. We also report WER reductions achieved through speech enhancement (which is quite poor) in Appendix K.

points. Limiting frame level features to the emission frame based on greedy decoding has a smaller effect: AP drops by 0.8 points on AMI and 0.2 points on CHiME-6.

5.2 Recourse

Figure 4 shows the WER reduction of PARDON and of the same pipeline driven by Peng et al. (2026) against the re-utterance burden, for spans of at most six words and $\beta = 0.05$ on AMI and CHiME-6. On AMI, PARDON removes 23% of the word errors with synthesized replies at a burden of 6 per minute, where Peng et al. needs 8, and 16% with close-talking replies at 6 per minute, where Peng et al. needs 9. On CHiME-6, whose noisy decode has about twice the WER of AMI (64% vs. 30%), the same reductions take higher burdens: PARDON removes 18% and 13% at 7 per minute, where Peng et al. needs 8 and 9.

Exceeding the zero-interruption N-best correction of HyPoradise (6% on AMI and 5% on CHiME-6) requires a burden of around 2 per minute on both corpora, with either kind of reply. At the highest burdens, PARDON with synthesized replies comes within 4 points of the N-best oracle (28% on AMI and 27% on CHiME-6), which no zero-interruption method can exceed with these hypotheses. Synthesized replies remove more errors than close-talking ones, by up to 7 points, likely because the close-talking recording of a span also carries disfluencies and crosstalk from the other speakers and is decoded with more errors. The ordering of the two scorers is the same with $\beta = 0.10$ and with spans of up to eight words (Appendix L).

Features	AMI			CHiME-6		
	AP	Recall at prec.		AP	Recall at prec.	
		50	70		50	70
CF-Gain (all features)	**56.5**	**61.7**	32.5	**66.7**	80.3	**49.4**
Emission frame only	55.7	60.5	**33.4**	66.5	**81.4**	46.9
w/o layer 13	54.2	58.7	24.1	65.7	78.4	46.3
w/o joint scalars	54.8	59.5	28.2	63.7	77.6	41.0

Table 4: Feature ablations of the CF-Gain Scorer on the evaluation sets of AMI and CHiME-6. *Emission frame only*: uses encoder features only from the greedy emission frame. *w/o layer 13*: only the final encoder layer is used. *w/o joint scalars*: the scalar features derived from the joint network are removed. All numbers are in percent; best per column in bold.

6 Conclusion

We introduced PARDON, which improves streaming recognition under noise by asking the speaker to repeat the stretches it has likely misheard. Its CF-Gain Scorer reads its evidence from the transducer's own alignment lattice, is trained on soft labels that marginalize over the alignment between the noisy and the counterfactual clean decode, and estimates how many words a repetition would recover. On four corpora, it ranks erroneous tokens above entropy-based and learned confidence estimators, and it transfers unchanged to another corpus of the same genre. With these scores, PARDON removes 23% of the word errors on far-field AMI meetings with synthesized replies, and 16% with close-talking replies, at a re-utterance burden of under 6 per minute, more than both an N-best LLM corrector and the same pipeline driven by Peng et al. (2026). Evaluating clarification with real speakers instead of simulated replies is the natural next step.

Limitations

We study a single recourse mechanism, targeted repetition. In realistic interactions, users may respond in many other ways, for instance by paraphrasing, correcting only part of the span or adding context, which would make the LLM corrector's task harder, and our replies are simulated rather than collected from real speakers. The scoring step is designed for one family of streaming recognizers, RNN transducers, whose alignment lattice it reads; other architectures would need a different source of token-level evidence. Finally, we focus on errors caused by far-field recording, because the parallel close-talking audio our supervision needs is readily available in such corpora; the same counterfactual supervision could be applied to other kinds of ASR errors wherever a cleaner rendering of the same speech can be obtained.

Ethical Considerations

All four corpora (AMI, NOTSOFAR-1, CHiME-6 and DiPCo) are publicly released for research and were recorded with the consent of their participants; we use them under their licenses and do not redistribute any audio. No new data was collected from people: speaker replies are simulated with the existing close-talking recordings or with synthesized speech.

A deployed system raises two risks. First, recognition errors are more frequent for speakers whose accents or speaking styles are under-represented in training data, so such speakers may be interrupted more often and bear more of the burden of correction; we have not measured this, and it should be evaluated before deployment. Second, the LLM corrector can insert words the speaker did not say, which matters when transcripts serve as records, such as meeting minutes; a deployed system should mark corrected spans.

References

AMI Consortium. 2005. AMI corpus official partitions. https://groups.inf.ed.ac.uk/ami/corpus/datasets.shtml. Accessed 2026-10-01.

Dan Bohus and Alexander I. Rudnicky. 2005. Sorry, I didn't catch that! – an investigation of non-understanding errors and recovery strategies. In *Proceedings of the 6th SIGdial Workshop on Discourse and Dialogue*, pages 128–143, Lisbon, Portugal. Special Interest Group on Discourse and Dialogue (SIGdial).

Jean Carletta, Simone Ashby, Sebastien Bourban, Mike Flynn, Mael Guillemot, Thomas Hain, Jaroslav Kadlec, Vasilis Karaiskos, Wessel Kraaij, Melissa Kronenthal, Guillaume Lathoud, Mike Lincoln, Agnes Lisowska, Iain McCowan, Wilfried Post, Dennis Reidsma, and Pierre Wellner. 2006. The AMI meeting corpus: A pre-announcement. In *Machine Learning for Multimodal Interaction (MLMI 2005)*, volume 3869 of *Lecture Notes in Computer Science*, pages 28–39, Berlin, Heidelberg. Springer.

Chen Chen, Yuchen Hu, Chao-Han Huck Yang, Sabato Marco Siniscalchi, Pin-Yu Chen, and Eng Siong Chng. 2023. HyPoradise: An open baseline for generative speech recognition with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*. ArXiv:2309.15701.

Samuele Cornell, Taejin Park, He Huang, Christoph Boeddeker, Xuankai Chang, Matthew Maciejewski, Matthew Wiesner, Paola Garcia, and Shinji Watanabe. 2024. The CHiME-8 DASR challenge for generalizable and array agnostic distant automatic speech recognition and diarization. In *Proc. 8th International Workshop on Speech Processing in Everyday Environments (CHiME 2024)*, pages 1–6.

Richard Durbin, Sean R. Eddy, Anders Krogh, and Graeme Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.

Google DeepMind. 2026. Gemini 3.8 flash model card. https://deepmind.google/models/model-cards/gemini-3-8-flash/. Accessed October 2026.

Xinlu He, Yiwen Guan, Badrivishal Paurana, Pitipat Kongsomjit, Zilin Dai, and Jacob Whitehill. 2025. Interactive in-meeting speaker correction with human feedback. *arXiv preprint arXiv:2509.18377*.

hexgrad. 2025. Kokoro-82M. https://huggingface.co/hexgrad/Kokoro-82M. Model card.

Yuchen Hu, Chen Chen, Chao-Han Huck Yang, Ruizhe Li, Chao Zhang, Pin-Yu Chen, and Eng Siong Chng. 2024. Large language models are efficient learners of noise-robust speech recognition. In *International Conference on Learning Representations (ICLR)*. ArXiv:2401.10446.

Hui Jiang. 2005. Confidence measures for speech recognition: A survey. *Speech Communication*, 45(4):455–470.

Federico Landini, Ján Profant, Mireia Diez, and Lukáš Burget. 2022. Bayesian HMM clustering of x-vector sequences (VBx) in speaker diarization: Theory, implementation and analysis on standard tasks. *Computer Speech & Language*, 71:101254.

Aleksandr Laptev and Boris Ginsburg. 2022. Fast entropy-based methods of word-level confidence estimation for end-to-end automatic speech recognition. In *IEEE Spoken Language Technology Workshop (SLT)*.

Yichong Leng, Xu Tan, Linchen Zhu, Jin Xu, Renqian Luo, Linquan Liu, Tao Qin, Xiang-Yang Li, Ed Lin, and Tie-Yan Liu. 2021. FastCorrect: Fast error correction with edit alignment for automatic speech recognition. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*.

Rao Ma, Mengjie Qian, Potsawee Manakul, Mark Gales, and Kate Knill. 2023. Can generative large language models perform ASR error correction? *arXiv preprint arXiv:2307.04172*.

NVIDIA. 2026. Nemotron speech streaming English 0.6B. `https://huggingface.co/nvidia/nemotron-speech-streaming-en-0.6b`. Model card.

E. S. Page. 1954. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115.

Yizhou Peng, Ziyang Ma, Changsong Liu, Yi-Wen Chao, Xie Chen, and Eng Siong Chng. 2026. Proactive for uncertainty: Cause-aware error diagnosis and interactive clarification for spoken dialogue systems. *Preprint*, arXiv:2605.25404. Accepted to Findings of EMNLP 2026.

David Qiu, Yanzhang He, Qiujia Li, Yu Zhang, Liangliang Cao, and Ian McGraw. 2021. Multi-task learning for end-to-end ASR word and utterance confidence with deletion prediction. In *Interspeech 2021*, pages 4074–4078.

Anton Ragni, Qiujia Li, Mark J. F. Gales, and Yu Wang. 2018. Confidence estimation and deletion prediction using bidirectional recurrent neural networks. In *IEEE Spoken Language Technology Workshop (SLT)*.

Nagarathna Ravi, Thishyan Raj T, Ravi Teja Chaganti, and Vipul Arora. 2025. ASR confidence estimation using true class lexical similarity score. In *Interspeech 2025*, pages 3658–3662.

Dima Rekesh, Nithin Rao Koluguri, Samuel Kriman, Somshubra Majumdar, Vahid Noroozi, He Huang, Oleksii Hrinchuk, Krishna Puvvada, Ankur Kumar, Jagadeesh Balam, and Boris Ginsburg. 2023. Fast conformer with linearly scalable attention for efficient speech recognition. In *IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*.

Hendrik Schröter, Alberto N. Escalante-B., Tobias Rosenkranz, and Andreas Maier. 2023. DeepFilterNet: Perceptually motivated real-time speech enhancement. In *Interspeech 2023*, pages 2008–2009.

Silero Team. 2021. Silero VAD: Pre-trained enterprise-grade voice activity detector. `https://github.com/snakers4/silero-vad`.

Gabriel Skantze. 2007. *Error Handling in Spoken Dialogue Systems: Managing Uncertainty, Grounding and Miscommunication*. Ph.D. thesis, KTH Royal Institute of Technology, Stockholm, Sweden.

David Snyder, Guoguo Chen, and Daniel Povey. 2015. MUSAN: A music, speech, and noise corpus. ArXiv:1510.08484.

Maarten Van Segbroeck, Ahmed Zaid, Ksenia Kutsenko, Cirenia Huerta, Tinh Nguyen, Xuewen Luo, Björn Hoffmeister, Jan Trmal, Maurizio Omologo, and Roland Maas. 2020. DiPCo – dinner party corpus. In *Interspeech 2020*, pages 434–436.

Alon Vinnikov, Amir Ivry, Aviv Hurvitz, Igor Abramovski, Sharon Koubi, Ilya Gurvich, Shai Peer, Xiong Xiao, Benjamin Martinez Elizalde, Naoyuki Kanda, Xiaofei Wang, Shalev Shaer, Stav Yagev, Yossi Asher, Sunit Sivasankaran, Yifan Gong, Min Tang, Huaming Wang, and Eyal Krupka. 2024. NOTSOFAR-1 challenge: New datasets, baseline, and tasks for distant meeting transcription. In *Interspeech 2024*, pages 5003–5007.

Mingqiu Wang, Hagen Soltau, Laurent El Shafey, and Izhak Shafran. 2021. Word-level confidence estimation for RNN transducers. In *IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*.

Shinji Watanabe, Michael Mandel, Jon Barker, Emmanuel Vincent, Ashish Arora, Xuankai Chang, Sanjeev Khudanpur, Vimal Manohar, Daniel Povey, Desh Raj, David Snyder, Aswin Shanmugam Subramanian, Jan Trmal, Bar Ben Yair, Christoph Boeddeker, Zhaoheng Ni, Yusuke Fujita, Shota Horiguchi, Naoyuki Kanda, and 2 others. 2020. CHiME-6 challenge: Tackling multispeaker speech recognition for unsegmented recordings. In *Proc. 6th International Workshop on Speech Processing in Everyday Environments (CHiME 2020)*, pages 1–7.

Chao-Han Huck Yang, Yile Gu, Yi-Chieh Liu, Shalini Ghosh, Ivan Bulyko, and Andreas Stolcke. 2023. Generative speech recognition error correction with large language models and task-activating prompting. In *IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*.

Contents of the Appendix

A Transducer Alignment Lattice: Definitions **11**

B Token Features **12**

C PARDON Algorithm **13**
C.1 Online spans with the CUSUM algorithm 13

D Data Preparation **13**
D.1 Corpora 13
D.2 AMI 14
D.3 NOTSOFAR-1, CHiME-6 and DiPCo 15
D.4 Speech Synthesis 15

E Recourse Details **16**

F Prompts **16**
F.1 Corrector 16
F.2 N-best Correction 17

G Implementation of the Learned Confidence Estimators **17**

H Pair-HMM Channel **18**

I Recourse Examples **18**

J Out-of-Domain Generalization with Matched Training Size **20**

K Speech Enhancement Baseline **20**

L Additional Recourse Results **20**

M Tuning Cost Parameters **20**

N Training Details **21**

A Transducer Alignment Lattice: Definitions

For an utterance $\hat{X}$ of T frames $(\hat{x}_1, \ldots, \hat{x}_T)$ and a transcript $\hat{Y}$ of N tokens $(\hat{y}_1, \ldots, \hat{y}_N)$, an alignment is a monotonic path through a $T \times (N{+}1)$ grid whose node (t, j) is the state in which the first t frames $\hat{x}_1, \ldots, \hat{x}_t$ have been processed, $\hat{y}_1, \ldots, \hat{y}_{j-1}$ have been emitted, and the decoder waits to emit $\hat{y}_j$; row $N{+}1$ follows the last token. We consider only the paths that run from $(1, 1)$ to $(T, N{+}1)$. They differ only in *when* each token is emitted, and the probability of the transcript is the total probability of all of them,

$$P(\hat{Y} \mid \hat{X}) \;=\; \sum_{\text{paths } (1,1)\to(T,N+1)} P(\text{path} \mid \hat{X}). \tag{5}$$

At each node the model produces a distribution over $\mathcal{V} \cup \{\epsilon\}$, where $\mathcal{V}$ is the vocabulary of the model and ϵ the blank,

$$P(\cdot \mid t, j) = \operatorname{softmax}\big(\operatorname{Joint}(h[t], g[j])\big), \tag{6}$$

where $h[t]$ is the encoder state at frame t, conditioned on the audio up to that frame, and $g[j]$ the predictor state, conditioned on $\hat{y}_1, \ldots, \hat{y}_{j-1}$.

Everything that follows is computed on a lattice *restricted* to the 1-best transcript $\hat{Y}$ obtained by greedy decoding. Although the model assigns probability to every element of $\mathcal{V}$ at every node, the restricted lattice admits only the token that continues $\hat{Y}$. Only two transitions therefore leave (t, j): emitting ϵ, which advances the frame, $(t, j) \to (t{+}1, j)$, and emitting $\hat{y}_j$, which advances the token index, $(t, j) \to (t, j{+}1)$, with probabilities

$$b[t, j] = P(\epsilon \mid t, j), \quad e[t, j] = P(\hat{y}_j \mid t, j). \tag{7}$$

The forward variable $\alpha[t, j]$ is the probability of reaching (t, j) and the backward variable $\beta[t, j]$ the probability of completing the alignment from it:

$$\alpha[1, 1] = 1, \tag{8}$$

$$\alpha[t, j] = \alpha[t{-}1, j]\, b[t{-}1, j] + \alpha[t, j{-}1]\, e[t, j{-}1], \tag{9}$$

$$\beta[T, N{+}1] = b[T, N{+}1], \tag{10}$$

$$\beta[t, j] = \beta[t{+}1, j]\, b[t, j] + \beta[t, j{+}1]\, e[t, j], \tag{11}$$

with out-of-lattice terms taken as zero, giving $Z = P(\hat{Y} \mid \hat{X}) = \alpha[T, N{+}1]\, b[T, N{+}1] = \beta[1, 1]$.

Emission probability. $q[t, j]$ is the posterior probability that token $\hat{y}_j$ is emitted at frame t, given that $\hat{Y}$ is the final transcript; it is the occupancy of the label transition out of (t, j):

$$q[t, j] \;=\; \frac{\alpha[t, j]\; e[t, j]\; \beta[t, j{+}1]}{Z}. \tag{12}$$

Dwell probability. $r[t, j]$ is the posterior probability, given that $\hat{Y}$ is the final transcript, that the decoder advances from frame t to frame $t{+}1$ while

waiting to emit $\hat{y}_j$, i.e. in the gap between $\hat{y}_{j-1}$ and $\hat{y}_j$ (the frame at which $\hat{y}_{j-1}$ is emitted included); it is the occupancy of the blank transition out of (t, j):

$$r[t, j] \;=\; \frac{\alpha[t, j]\; b[t, j]\; \beta[t{+}1, j]}{Z}. \qquad (13)$$

Normalization. The two quantities normalize along different axes. Each token is emitted exactly once in any alignment, so $\sum_t q[t, j] = 1$ and $q[\cdot, j]$ is a distribution over frames. In contrast, because each frame is advanced exactly once, $\sum_j r[t, j] = 1$: $r[\cdot, j]$ instead sums to the expected number of blank frames between $\hat{y}_{j-1}$ and $\hat{y}_j$, which may exceed one or approach zero. When pooling frame-level features to token level we therefore divide $r[\cdot, j]$ by its sum over frames.

Streaming estimation. The posteriors above use the whole utterance. In PARDON the posteriors of token $\hat{y}_j$ are computed when the greedy decoder emits it, at frame t_j. Let N_j be the number of tokens the decoder has emitted by frame t_j (at least j, more if further tokens are emitted at the same frame). The truncated backward variable β_j follows the recursion above on frames $1, \ldots, t_j$ and rows $1, \ldots, N_j{+}1$, with the terminal condition

$$\beta_j[t_j{+}1, N_j{+}1] = 1, \qquad (14)$$

$$\beta_j[t_j{+}1, i] = 0 \quad (i \le N_j), \qquad (15)$$

so that every path has emitted the N_j known tokens by the cut, and $Z_j = \beta_j[1, 1]$. The forward variable needs no change, since it depends only on earlier frames. The posteriors of token j are then

$$q_j[t, j] = \alpha[t, j]\, e[t, j]\, \beta_j[t, j{+}1] \,/\, Z_j, \qquad (16)$$

$$r_j[t, j] = \alpha[t, j]\, b[t, j]\, \beta_j[t{+}1, j] \,/\, Z_j, \qquad (17)$$

for $t \le t_j$, and zero for later frames. When $t_j = T$ this is the full lattice. The end-of-sequence position $\hat{y}_{N+1}$ is never emitted, so it has no emission frame; its posteriors are computed once the utterance has ended, on the full lattice, from row $N{+}1$ (the frames after the last token).

Band. Both recursions are further restricted to the nodes (t, j) with

$$t_{j-1} - \delta \;\le\; t \;\le\; t_{j+k}, \qquad (18)$$

where the lower bound is 1 for $j = 1$ and the upper bound is T when $j + k > N$; α and β_j are zero outside the band. Since node (t, j) is reached only once $\hat{y}_{j-1}$ has been emitted, a path may thus emit each token at most δ frames before the decoder did, and may fall at most k tokens behind it. We use $k = 3$ and $\delta = 16$ frames.

B Token Features

The input $\mathbf{f}_j$ of the CF-Gain Scorer concatenates four pooled encoder vectors (the emission and dwell views of §3.2.1 at the intermediate and the final encoder layer), the predictor state $g[j]$, and the 27 scalars below: 20 computed from the joint network's output distribution (1–20) and 7 from the voice-activity probability $v[t]$ (21–27). Every feature is computed from the streaming posteriors of §3.2.1, so it uses only frames up to the emission of $\hat{y}_j$; features are standardized with training-set statistics.

Notation. $P_t = P(\cdot \mid t, j)$ is the output distribution over $\mathcal{V} \cup \{\epsilon\}$ at node (t, j), and $e[t, j]$ and $b[t, j]$ are the probabilities of the emitted token and of the blank (Appendix A). Let $o[t, j] = 1 - e[t, j] - b[t, j]$ be the probability of any other token. The emission frames of $\hat{y}_j$ are $\mathcal{T}_j^{\text{emit}} = \{t : q[t, j] > 0.01\}$, and its dwell frames are $\mathcal{T}_j^{\text{gap}} = \{t : r[t, j] \ge 0.01 \max_{t'} r[t', j]\}$, the frames of the gap before it; $\tilde{r}[t, j]$ is the normalized dwell posterior. The voice-activity probability $v[t]$ is the unweighted mean of the Silero VAD (Silero Team, 2021) outputs of the three 32 ms chunks that overlap the 80 ms encoder frame, including a chunk that overlaps it only partly. For a distribution p, $H(p) = -\sum_k p_k \log p_k$ is its Shannon entropy and $S_\alpha(p) = (1 - \sum_k p_k^\alpha)/(\alpha - 1)$ its Tsallis entropy. A statistic over an empty frame set is 0.

Uncertainty at emission (5). How spread out the model's output is when it emits $\hat{y}_j$, averaged over the frames at which it may have been emitted:

(1) $\sum_t q[t, j]\, H(P_t)$
(2–4) $\sum_t q[t, j]\, S_\alpha(P_t)$ for $\alpha = 0.5, 2, 4$
(5) $\sum_t q[t, j] \log e[t, j]$

The Shannon entropy weighs all alternatives; the Tsallis entropies, which Laptev and Ginsburg (2022) found well suited to transducer confidence, shift the weight from the many low-probability alternatives ($\alpha < 1$) to the few strongest competitors ($\alpha > 1$). The expected log-probability (5) is the usual token confidence.

Probability of other tokens (3). The probability the model gives to tokens other than $\hat{y}_j$ and the blank, averaged under three weightings:

(6) $\sum_t q[t,j]\, o[t,j]$, at the emission
(7) $\sum_t \tilde{r}[t,j]\, o[t,j]$, over the gap
(8) $\sum_t w[t,j]\, o[t,j] \,/\, \sum_t w[t,j]$, over both, with $w = q[\cdot,j] + r[\cdot,j]$

At the emission, mass on other tokens indicates that a different word may have been spoken (a substitution). In the gap, where the decoder emitted only blanks, it indicates that the model considered emitting a word there and did not, the trace a deleted word leaves.

Frame statistics (12). The weighted averages above can hide a single frame at which the model strongly preferred something else. We therefore also take, without weighting, the mean, maximum and 90th percentile of two per-frame values over each frame set:

(9–11) $\log e[t,j]$ over $t \in \mathcal{T}_j^{\text{emit}}$
(12–14) $o[t,j]$ over $t \in \mathcal{T}_j^{\text{emit}}$
(15–17) $\log e[t,j]$ over $t \in \mathcal{T}_j^{\text{gap}}$
(18–20) $o[t,j]$ over $t \in \mathcal{T}_j^{\text{gap}}$

Over the emission frames, they describe how confidently and how consistently $\hat{y}_j$ was preferred. Over the gap, (15–17) show whether $\hat{y}_j$ was already likely before it was emitted, and (18–20) whether some other token was, at any point in the gap.

Voice activity (7). Whether the frames contain speech at all, independently of the recognizer:

(21) $\sum_t \tilde{r}[t,j]\, v[t]$, over the gap
(22–24) mean, maximum and 90th percentile of $v[t]$ over $t \in \mathcal{T}_j^{\text{emit}}$
(25–27) mean, maximum and 90th percentile of $v[t]$ over $t \in \mathcal{T}_j^{\text{gap}}$

Speech in a gap through which the decoder emitted only blanks suggests that words were spoken there and lost; little speech around an emitted token suggests that it was produced from noise rather than from a word.

The end-of-sequence position $\hat{y}_{N+1}$ is treated as a token emitted at the last frame: its emission frame set is that frame, its dwell frames are those after $\hat{y}_N$, and since only the blank can be emitted from row $N{+}1$, the probability of the final blank takes the place of e, with $e[t,N{+}1] = b[t,N{+}1]$ and $o[t,N{+}1] = 1 - b[t,N{+}1]$.

C PARDON Algorithm

Algorithm 1 summarizes the training of the CF-Gain Scorer (§3.2) and the streaming inference with span selection and recourse (§3.3).

C.1 Online spans with the CUSUM algorithm

Re-uttering word k yields a net gain of $\bar{m}_k - c_R$, and a span of words warrants an interruption when these gains sum to at least c_I. We track the gains with a cumulative sum (CUSUM; Page, 1954):

$$C_k = \max\big(0,\ C_{k-1} + \bar{m}_k - c_R\big), \quad C_0 = 0. \tag{19}$$

C_k is the largest total gain of a span of words that ends at word k and starts after the last interruption. Whenever the cumulative gain becomes positive, we track its current value C_k, its maximum P during the positive run and the word where P was reached. Once $P - C_k \geq c_I$, we interrupt and select the span from the first word of the positive run to where P was reached. Words after the peak are not part of the span; they only ensure that we do not interrupt while the speech remains degraded. After the interruption, the cumulative gain resets to zero. If C_k returns to zero before $P - C_k$ reaches c_I, no span is formed. If the turn ends first, we interrupt if $P \geq c_I$.

D Data Preparation

D.1 Corpora

The AMI Meeting Corpus (Carletta et al., 2006) contains about 100 hours of four-person meetings recorded by headset microphones and far-field arrays. NOTSOFAR-1 (Vinnikov et al., 2024) contains office meetings recorded in many different conference rooms by several commercial devices at once, with close-talking microphones worn by the participants; we keep one device per meeting, so that no speech appears twice. CHiME-6 (Watanabe et al., 2020) contains four-person dinner parties recorded in private homes by distant microphone arrays and by binaural microphones worn by each speaker. DiPCo (Van Segbroeck et al., 2020) contains ten dinner-party sessions of four participants, recorded by five distant seven-microphone devices and by a close-talking microphone per speaker. In all four, the noisy input is the first channel of the first far-field array. Speaking turns in the two dinner-party corpora are much shorter than in the meeting corpora.

Algorithm 1 PARDON end to end. TRAIN (offline) fits the CF-Gain Scorer p_θ on a dataset $\mathcal{D}$ of noisy and clean renderings of the same utterances; INFER runs alongside the streaming decode of $\mathcal{M}_\theta$. The pair HMM has parameters $\Phi = (\pi, \gamma, \eta, \rho)$: the probability that a noisy word is an error (π), that a run of deleted words continues (γ), that the error after a deletion is its anchor (η), and that a non-anchor error is an insertion rather than a substitution (ρ) (Appendix H); A is an alignment of $\hat{Y}_i$ to Y_i^c, and $E_j(A)$, $D_j(A)$ mark that $\hat{y}_j$ is an error and that words are deleted before it under A (§3.2.2). t_j is the frame at which $\hat{y}_j$ is emitted, and s_j the LSTM state. In CUSUM, C is the running sum of Eq. (19), a the first word of the open span ($\perp$ if none), and P, p the peak of C and its word.

procedure TRAIN($\mathcal{D} = \{(\hat{X}_i, Y_i, X_i^c)\}_{i=1}^N, \mathcal{M}_\theta$)
  for all $i = 1, \ldots, N$ **do**
    $\hat{Y}_i, Y_i^c \leftarrow$ word decodes of $\hat{X}_i, X_i^c$ by $\mathcal{M}_\theta$
    $A_i^0 \leftarrow$ hard alignment (longest common runs)
  end for
  $\Phi \leftarrow$ event rates under $\{A_i^0\}$, e.g. $\rho \leftarrow \frac{\#\text{ins}}{\#\text{ins}+\#\text{sub}}$
  for 10 iterations **do** ▷ EM on the training pairs
    E: $n_x \leftarrow \sum_i \mathbb{E}_{P(A \mid Y_i^c, \hat{Y}_i; \Phi)}[\#x \text{ in } A]$
    by forward–backward, for each event x:
    error or correct word, deletion continued
    or ended, anchor or not, insertion or substitution
    M: $\pi \leftarrow \frac{n_{\text{err}}}{n_{\text{err}}+n_{\text{cor}}}$, $\gamma \leftarrow \frac{n_{\text{cont}}}{n_{\text{cont}}+n_{\text{end}}}$,
      $\eta \leftarrow \frac{n_{\text{anc}}}{n_{\text{anc}}+n_{\text{non-anc}}}$, $\rho \leftarrow \frac{n_{\text{ins}}}{n_{\text{ins}}+n_{\text{sub}}}$
  end for
  freeze Φ
  for all i and tokens j of $\hat{Y}_i$ **do**
    $\tilde{e}_{ij} \leftarrow \sum_A P(A \mid Y_i^c, \hat{Y}_i)\, E_j(A)$
    $\tilde{d}_{ij} \leftarrow \sum_A P(A \mid Y_i^c, \hat{Y}_i)\, D_j(A)/\tilde{e}_{ij}$
    $\tilde{\ell}_{ij} \leftarrow$ number of words deleted before $\hat{y}_j$ in A_i^0
    $\mathbf{f}_{ij} \leftarrow$ FEATURES(j)
  end for
  $w, w_d \leftarrow$ ratios of negative to positive labels
  train p_θ on $\{(\mathbf{f}_{ij}, \tilde{e}_{ij}, \tilde{d}_{ij}, \tilde{\ell}_{ij})\}$ with Eq. (3)
  Platt-scale e_j on dev, and d_j on its error tokens
end procedure

procedure FEATURES(j) ▷ frames $t \le t_j$ only
  $q[\cdot, j], r[\cdot, j] \leftarrow$ banded forward–backward
  $\tilde{r}[t, j] \leftarrow r[t, j] / \sum_{t'} r[t', j]$
  pool $h^{(13)}, h^{(24)}, v$, joint scalars under $q[\cdot, j], \tilde{r}[\cdot, j]$
  return the pooled features and $g[j]$
end procedure

procedure INFER(stream $\hat{X}$; c_R, c_I, W)
  $\hat{Y}' \leftarrow [\,]$; $C \leftarrow 0$; $a \leftarrow \perp$; $s_0 \leftarrow \mathbf{0}$
  for all tokens $\hat{y}_j$ emitted by $\mathcal{M}_\theta$, at frame t_j **do**
    $(e_j, d_j, \ell_j, s_j) \leftarrow p_\theta(\text{FEATURES}(j), s_{j-1})$
    $m_j \leftarrow e_j\,[(1 - d_j) + d_j\,\ell_j]$ ▷ Eq. (4)
    if $\hat{y}_j$ completes word k **then**
      append word k to $\hat{Y}'$
      CUSUM(k, $\max_{\hat{y}_i \in \text{word } k} m_i$)
    end if
  end for
  CUSUM($N{+}1$, m_{N+1}) ▷ end of sequence
  if $a \neq \perp$ and $P \ge c_I$ **then** RECOURSE($a..p$)
  return $\hat{Y}'$
end procedure

procedure CUSUM(k, $\bar{m}_k$)
  if $C = 0$ and $\bar{m}_k > c_R$ **then**
    $a \leftarrow k$; $P \leftarrow 0$ ▷ span opens
  end if
  $C \leftarrow \max(0,\, C + \bar{m}_k - c_R)$ ▷ Eq. (19)
  if $a \neq \perp$ and $C > P$ **then** $P \leftarrow C$; $p \leftarrow k$
  if $a \neq \perp$ and ($C = 0$ or $C \le P - c_I$) **then**
    if $P \ge c_I$ **then** RECOURSE($a..p$)
    $a \leftarrow \perp$; $C \leftarrow 0$
  end if
end procedure

procedure RECOURSE(span S)
  if $|S| > W$ **then** ▷ length cap
    $S \leftarrow$ its W consecutive words with max $\sum \bar{m}_k$
  end if
  $o_L, o_R \leftarrow$ anchor words on either side of S in $\hat{Y}'$
  ask $Q(o_L, o_R)$; record the reply audio X^r
  $Y^r \leftarrow$ decode of X^r by $\mathcal{M}_\theta$
  $Z \leftarrow \text{LLM}(\hat{Y}' \text{ with } S \text{ marked}, Q, Y^r)$
  replace the words of S in $\hat{Y}'$ by Z
end procedure

The corpora are cut into utterances in two ways: AMI has a time stamp for every word, so its long turns can be split between two words, whereas the other three corpora only time each segment, so their long turns cannot be split and are left out of training. Across all of them, training excludes turns too short to carry reliable labels (3 s for the meeting corpora and 1.5 s for the dinner parties), while development and evaluation keep every turn. Table 5 gives the resulting sizes.

D.2 AMI

Segments. Speaker turns are taken from the `only_words` references of the AMI diarization setup of Landini et al. (2022), which are derived from the AMI manual annotations (version 1.6.2): every annotated word is speech, consecutive words of the same speaker are merged into one turn, and turns are broken wherever the speaker pauses. The setup's meeting lists follow the full-corpus-ASR partition (136 training, 18 development and 16 evaluation meetings). Two meetings in the training set (IS1003b, IS1007d) do not have far-field recording in the corpus and are excluded. Each speaker's global identifier is mapped to their headset channel using the corpus metadata (`meetings.xml`).

Length criteria. Turns longer than 15 s are split into pieces of at most 15 s, cutting only between two consecutive words of the speaker, at the midpoint of the pause between them. For each piece, the cut is placed at the widest pause whose midpoint lies between 7.5 s and 15 s from the start of the piece, falling back to pauses closer to the start, and avoiding cuts that would leave a short remainder; if no word boundary is available, the piece is cut at 15 s. In the training set, turns and pieces shorter than 3 s are discarded: 6.37% of turns are split, and 34.78% of training utterances come from split turns. The 3 s threshold removes 66% of turns but only 16% of the speech, and merging consecutive turns of the same speaker across pauses of up to 1 s recovers less than 1% of the speech, so we do not merge turns. The development and evaluation sets keep turns of every length; in them 4.29% and 6.67% of turns are split, giving 9.33% and 13.96% of their utterances.

Reference transcripts. The reference of each utterance consists of the words of the speaker's manual transcript whose start time falls inside it. Words without a time stamp are included when the nearest time-stamped words on both sides are included. Non-word annotations (vocal sounds, gaps and disfluency markers) and tokens consisting only of punctuation are excluded.

Audio. The far-field and headset recordings are cut at the same boundaries, down-mixed to mono and resampled to 16 kHz.

D.3 NOTSOFAR-1, CHiME-6 and DiPCo

Sources. The three corpora are converted with the data-generation tools of the CHiME-8 distant ASR challenge (Cornell et al., 2024), which provide segment-level transcripts with a start time, an end time, a speaker and the words of each segment. For NOTSOFAR-1 we use the training (`240825.1_train`), development (`240825.1_dev1`) and evaluation (`240629.1_eval_small_with_GT`, 80 meetings) releases. Each NOTSOFAR-1 meeting appears once per recording device; we keep the sessions of a single device (device code `107`), present in every meeting. The far-field input is the first channel of the first array (`U01`), and the clean recording is the speaker's close-talking channel.

Segments. Consecutive segments of the same speaker are joined into one turn when the pause between them is at most 0.5 s, no other speaker's segment overlaps the pause, no segment removed under the criteria below lies inside it, and the joined turn is at most 15 s long. The transcripts have no word-level time stamps, so a turn cannot be split at a word boundary as in AMI; in the training set, turns longer than 15 s are therefore discarded, as are turns shorter than 3 s (NOTSOFAR-1) or 1.5 s (CHiME-6, DiPCo). The development and evaluation sets keep every turn, with turns outside these limits kept whole.

Reference transcripts. Annotation tags (laughter, noise, and similar) are removed from the reference. Segments whose reference marks words as inaudible, unintelligible or redacted are excluded from training, since the recording contains words that their reference lacks; they are kept, with the tag removed, in the development and evaluation sets, where they make up a substantial share of DiPCo. Segments with no words after tag removal are excluded from training; in development and evaluation they are kept, and enter the evaluation whenever their close-talking decode is not empty.

Cross-talk. The close-talking microphones of these corpora, and the binaural microphones of CHiME-6 in particular, also pick up nearby speakers. Words of another speaker that appear in the close-talking decode are then counted as words the far-field decode deleted, which adds noise to the labels of overlapping turns.

D.4 Speech Synthesis

The synthetic clean rendering of a training utterance is its reference transcript, read by Kokoro-82M (hexgrad, 2025). Punctuation-only tokens and the hyphens of partial words are removed, and fillers are kept. Each speaker has a fixed voice, drawn from five female and five male voices. In AMI, the voice matches the speaker's gender, which the speaker identifier encodes; NOTSOFAR-1, CHiME-6 and DiPCo do not annotate gender, and their speakers get a voice from the pooled list. The audio is resampled to 16 kHz, padded with 0.25 s of silence on each side, and mixed with an excerpt of the free-sound noise of MUSAN (Snyder et al., 2015) at an SNR drawn uniformly from 20 to 30 dB. The synthesized replies of Appendix E are rendered in the same way from the reference words of the span. Every rendering is seeded by the utterance, so it is reproducible.

	Train		Dev		Eval	
Corpus	Utt.	Hours	Utt.	Hours	Utt.	Hours
AMI	27,861	62.9	9,145	8.8	8,128	8.5
NOTSOFAR-1	3,533	6.2	9,493	5.5	16,557	11.0
CHiME-6	30,778	28.5	6,926	6.5	9,940	5.9
DiPCo	526	0.7	2,246	2.0	3,279	3.2

Table 5: Utterances and hours of speech per set after segmentation. Training excludes short and partly unintelligible turns; development and evaluation keep every turn.

E Recourse Details

Span boundaries. Each word k is scored by $\bar{m}_k$, the highest m_j among its tokens; tokens without a score (punctuation) are ignored. Spans are found by the CUSUM of §3.3.1 (Algorithm 1): a span starts at the word where C rises above zero and ends at the word where C peaked, and the interruption is raised at the first word at which C has fallen c_I below that peak. Spans are made of whole words, so a request never quotes part of a word. A span longer than W words is cut to the W consecutive words with the largest total score ($W = 6$ unless stated otherwise). The two words before the span form the left anchor and the two words after it the right anchor.

Start and end of the utterance. Near either edge of the utterance, a span may have fewer than two words on that side, so its anchor is cut short: the left anchor never extends before the first word and the right anchor never past the last. A span that reaches an edge has no anchor on that side and uses the corresponding one-sided template below. At the end of the utterance, the end-of-sequence position $\hat{y}_{N+1}$, which carries the deletions after the last word, is scored as one more word, so trailing deletions can also open or extend a span. A span still open at that point does not wait for C to fall: it triggers an interruption if its peak gain P is at least c_I and is dropped otherwise. We take the end of the utterance from the reference segmentation; a deployed system would need a turn-end detector instead.

Request templates. The template depends on which anchors exist, with L the (up to) two words before the span, R the (up to) two words after it, and S the words of the span in the noisy decode:

- both anchors: *"I didn't catch what you said between 'L' and 'R'. Could you repeat it?"*
- left anchor only: *"What did you say after 'L'?"*
- right anchor only: *"What did you say before 'R'?"*
- no anchor, as when the span covers a whole short utterance: *"Did you say 'S'?"*

Replies. The speaker's reply is simulated in two ways: the speaker's close-talking recording of the span, cut at its word times and widened by 400 ms on each side, or a synthesized rendering of its reference words (Appendix D.4). Either reply is decoded on its own by the same recognizer, starting from an empty predictor state; an empty reference gives an empty reply.

Correction. Each span is corrected by Gemini 3.8 Flash (Google DeepMind, 2026) at temperature 0, as described in §3.3.2. The prompt, and how the corrector's output replaces the words of the span, are given in Appendix F.1.

F Prompts

Placeholders are shown in angle brackets. All calls use temperature 0.

F.1 Corrector

The corrector is called once per span, in the order in which the interruptions are raised. Its user message contains the transcript decoded up to the interruption, with earlier spans already replaced by their corrections and the current span marked `[? ... ?]`, followed by the request and the reply. Only the text inside `<fix></fix>` is used, and it replaces the words of the span; every other word of the transcript is left as decoded. When the reply is empty, no call is made and the span's words are removed.

System

> You correct a live speech-recognition transcript of a meeting. The recognizer flagged a stretch it probably got wrong; it is marked [? like this ?]. The system asked the speaker to repeat that stretch, and the speaker's answer is given. The answer was also transcribed automatically, so it can contain recognition errors, and the speaker may have repeated a few words from just before or after the stretch.
>
> Write the corrected text for the marked stretch only:
> - base it on the answer where the answer is plausible in context;
> - leave out words of the answer that belong to the transcript outside the marks;
> - keep the transcript's style (case, punctuation);
> - if the marked words were not actually spoken, return nothing.
>
> Reply with the corrected stretch inside `<fix></fix>` and nothing else.

User

Transcript so far:
⟨*words before the span*⟩ [?⟨*span words*⟩?] ⟨*words after the span, up to the interruption*⟩
System asked: ⟨*request (Appendix E)*⟩
Speaker answered: "⟨*transcribed reply*⟩"

F.2 N-best Correction

The N-best baseline uses the instruction template of HyPoradise (Chen et al., 2023) in a single user turn, preceded by ten demonstrations in the same format. The best hypothesis is the greedy transcript, which every other method corrects; the other hypotheses are the four highest-scoring beam hypotheses that remain distinct from it and from each other after text normalization. The demonstrations are the longest development-set utterances of at most 40 words, after removing utterances whose greedy hypothesis is far from the reference (cross-talk in the reference). The answer is read after `### Response:`; an empty or overly long answer is replaced by the greedy transcript. The N-best oracle picks, for each utterance, the hypothesis of the same list closest to the reference.

System

You correct speech recognition transcripts. Follow the format of the examples exactly and reply with the corrected transcription only.

User

Below is a best-hypotheses that is transcribed from an automatic speech recognition system. Write a response to predict the true transcription using the tokens from other-hypotheses.
Example 1:
Best-hypothesis:
⟨*greedy transcript*⟩
Other-hypothesis:
⟨*hypothesis 2*⟩
...
⟨*hypothesis 5*⟩
Response:
⟨*reference*⟩
... (Examples 2–10)
Now the utterance to correct:
Best-hypothesis:
⟨*greedy transcript*⟩
Other-hypothesis:
⟨*hypothesis 2*⟩
...
⟨*hypothesis 5*⟩
Response:

G Implementation of the Learned Confidence Estimators

All four learned estimators of §4.2 read the same frozen transducer and are trained on the same corpus, partition and labels as the CF-Gain Scorer: the hard labels E_j from the close-talking decode, with early stopping on development-set average precision. Every subword token is scored, and a word error marks all of its tokens. Each estimator is made causal: token j, emitted at frame t_j, is scored only from the tokens up to j and the encoder frames up to t_j, as the CF-Gain Scorer is.

Qiu et al. (2021). Each token's input is its embedding and position, the decoder activation, its log posterior and the top-K log posteriors, read at its greedy emission point.

- Self-attention is masked to tokens up to j. Cross-attention covers only the 16 encoder frames ending at t_j, instead of the whole utterance.
- The deletion head predicts our deletion anchor, the token after the gap, as an auxiliary task. The utterance-confidence head is an auxiliary task during training only. The token score is the output of the main head.
- We omit the multi-hypothesis input, which needs complete beams.

Wang et al. (2021).

- The three frames around the emission, $t_j - 1, t_j, t_j + 1$, are replaced by the three frames ending at it, $t_j - 2, t_j - 1, t_j$, since $t_j + 1$ may fall in the next encoder chunk.
- Self-attention covers the past 30 tokens only.

TruCLeS (Ravi et al., 2025).

- The input is the same as for Wang et al. (2021), and the LSTMs are unidirectional instead of bidirectional.
- The confidence target is computed with the close-talking decode as the reference. The probability of the reference token is read under the noisy decode's prefix, since the streaming recognizer never sees the true one. The error score is one minus the predicted confidence.

Peng et al. (2026). We reimplement the detectors from the description in the paper.

- **Detectors.** We keep the perception-error detector, which scores each token from the input to the recognizer's joint network, and the deletion detector, which scores encoder frames. We omit the comprehension-error detector: it is trained on errors against the reference transcript, so it would learn errors that a re-utterance does not correct. We also omit the

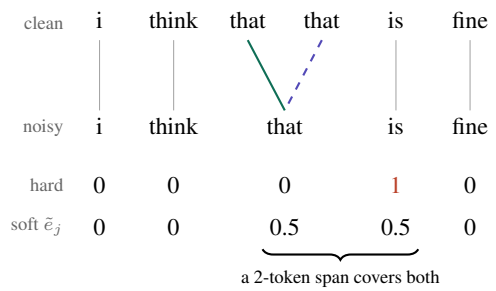


Figure 5: An ambiguous deletion. Two alignments explain the noisy decode equally well: the second *that* was deleted and *is* anchors the deletion (solid), or the first was deleted and the surviving *that* anchors it (dashed). The hard alignment labels only one token and treats the other as clean; the soft label splits the mass between them, so neither is penalized, and a span covering both carries the same label mass either way.

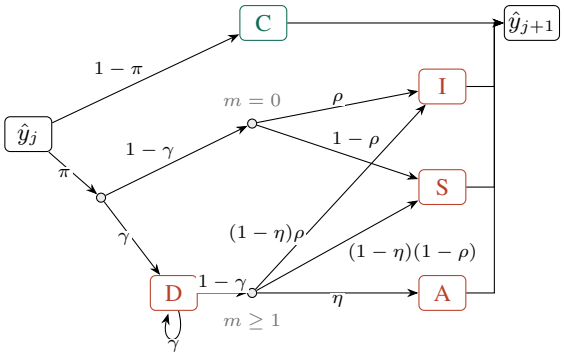


Figure 6: The pair-HMM channel at noisy token $\hat{y}_j$. A correct token (C) is written unchanged. An erroneous one is preceded by a run of deleted tokens (D, possibly empty) and is then an anchor (A, written unchanged; only after a deletion), an insertion (I, nothing written) or a substitution (S, a different token written). C has $E_j = 0$; A, I and S have $E_j = 1$.

distortion-type detector, whose labels require synthetic distortion conditions and whose output is used only by their own dialogue policy.

- **Deletion targets.** The paper does not specify where on the frame axis a deletion target is placed. We mark the frames the noisy decode spent waiting to emit the deletion's anchor token, those whose dwell posterior is at least 1% of its maximum (the final row of the lattice for a deletion after the last token). Deletion scores are pooled to a token over the blank frames that precede it, matching our anchoring convention.
- **Combination.** The scores of the two detectors are each mapped to percentiles fitted on the development set and combined by taking the maximum.

H Pair-HMM Channel

The soft labels of §3.2.2 come from a pair hidden Markov model (Durbin et al., 1998) that reads the noisy decode $\hat{Y}$ token by token and writes the clean decode Y^c (Figure 6). Figure 5 shows the case it is built for: a deletion that two alignments explain equally well.

Each noisy token carries a latent label $E_j \in \{0, 1\}$, and four parameters govern the channel: the probability that a token is in error (π), that a run of deleted tokens continues (γ), that an erroneous token is the anchor of a preceding deletion (η), and that a non-anchor error is an insertion rather than a substitution (ρ). They are fit by expectation maximization on the training pairs, initialized from the counts of the hard alignment, and frozen after ten iterations (Algorithm 1). The forward–backward pass of the fitted model gives the posterior $\tilde{e}_j = P(E_j = 1 \mid Y^c, \hat{Y})$ for every noisy token, including the end-of-sequence position, and in the same way the posterior that $\hat{y}_j$ is preceded by deleted tokens (the D state of Figure 6); dividing it by $\tilde{e}_j$ gives $\tilde{d}_j$. The number of deleted words $\tilde{\ell}_j$ is read from the hard alignment. Wherever the two decodes determine the alignment, $\tilde{e}_j$ equals the hard label E_j. The channel runs on normalized words, and its labels are mapped to subword tokens as described in §N.

I Recourse Examples

Substitution repaired

Decode	Yeah, **[we get behind schedule yeah]** pretty quick like you say do we want to set a date for interim prototype?
Request	I didn't catch what you said between 'Yeah' and 'pretty quick'. Could you repeat it?
Reply	It'll get behind schedule, yeah.
Corrected	Yeah, it'll get behind schedule yeah pretty quick like you say do we want to set a date for interim prototype?
Reference	Yeah it'll get behind schedule yeah pretty quick Like you're saying yeah Well actually do we wanna set a date for interim prototype

Substitution repaired

Decode	**[We have]** the number buttons, I guess.
Request	What did you say before 'the number'?
Reply	Leave out
Corrected	Leave out the number buttons, I guess.
Reference	leave out the number buttons I guess

Deleted words restored

Decode Okay, is that how it works in the other ones in like the other windows like various like um **[yeah topic]** or
Request I didn't catch what you said between 'like um' and 'or'. Could you repeat it?
Reply If we're in the topic
Corrected Okay, is that how it works in the other ones in like the other windows like various like um if we're in the topic or
Reference Okay Is that how it works in the other ones In like the other windows like the various like um if we're in the topic Or

Deleted words restored

Decode **[This is not really]** like for the group you were making this all really require a lot of marketing.
Request What did you say before 'like for'?
Reply It's pretty different, but this is not really
Corrected It's pretty different, but this is not really like for the group you were making are remote, this will really require a lot of marketing.
Reference It's pretty different But this is not really like for the group we were making our remote This will really require a lot of marketing

Inserted words removed

Decode Um, **[actually this]** first thing I just sort of did a really sort of brief um yeah version of what we have to do
Request I didn't catch what you said between 'Um' and 'first thing'. Could you repeat it?
Reply Actually
Corrected Um, actually first thing I just sort of did a really sort of brief um yeah version of what we have to do
Reference Um actually first thing I just sorta did a really sorta brief um yeah version of what we have to do Um actually I can just take that if you want

Partly repaired

Decode Nice and pink fluorescent banana color or whatever you know **[make a banana chip on you]** for example, you know, but the question is really who is our target group?
Request I didn't catch what you said between 'you know' and 'for example'. Could you repeat it?
Reply Yeah.
Corrected Nice and pink fluorescent banana color or whatever you know for example, you know, but the question is really who is our target group?
Reference nice and pink fluorescent banana colour or whatever you know yeah for example you know But the question is really who is our target group

Partly repaired

Decode **[Is]** there anything you would like to to ask me about the findings before I?
Request What did you say before 'there anything'?
Reply I know. Okay, well I hope that's a clear to the team is
Corrected I know. Okay, well I hope that's clear to the team. Is there anything you would like to to ask me about the findings before I?
Reference I know Okay Well I hope that's uh clear to the team Is there anything you would like to to ask me about the findings before I

False alarm, transcript unchanged

Decode Maybe we should just have two, but yeah, if **[he finds them then that's cool]** if he finds the people um.
Request I didn't catch what you said between 'yeah, if' and 'if he'. Could you repeat it?
Reply He finds them then that's cool.
Corrected Maybe we should just have two, but yeah, if he finds them then that's cool if he finds the people um.
Reference yeah Three or four Oh God Um but yeah if he finds them then that's cool If he finds the people M um

False alarm, transcript unchanged

Decode **[All these changes are not really]** okay with me, but since
Request What did you say before 'okay with'?
Reply All these changes are not really.
Corrected All these changes are not really okay with me, but since
Reference all these changes are not really okay with me But since

False alarm, error introduced

Decode Um like we have the the right click menu **[menu]** right clicking but instead of like have a default double click so there's so it'll be a choice
Request I didn't catch what you said between 'click menu' and 'right clicking'. Could you repeat it?
Reply Menu.
Corrected Um like we have the the right click menu right clicking but instead of like have a default double click so there's so it'll be a choice from the menu
Reference Um like we have the the right-click menu You'll have the menu right-click it but instead of like have a default double-click so if there's a de so it'll be a choice from the menu but yeah exactly Yeah

False alarm, error introduced

Decode **[There something like that,]** but that's that sounds a lot cheaper to me.
Request What did you say before 'but that's'?
Reply There's something like that.
Corrected There's something like that, but that's that sounds a lot cheaper to me.
Reference there something like that but that's that sounds a lot cheaper to me

Failure: the reply was misrecognized

Decode Yeah **[text yeah yeah yeah cause that's]** the like declares a variable isn't and then yeah because
Request I didn't catch what you said between 'Yeah' and 'the like'. Could you repeat it?
Reply Yeah yeah yeah cause that
Corrected Yeah yeah yeah cause that the like declares a variable isn't and then yeah because
Reference Yeah Yeah yeah 'cause that's the uh the declares a variable doesn't it and then they keep Okay

Failure: the reply was misrecognized

Decode	**[The whistle]** so we can financially voice recognition is not feasible we could go for a whistle.
Request	What did you say before 'so we'?
Reply	For hell yeah, the whistle.
Corrected	The whistle so we can financially voice recognition is not feasible we could go for a whistle.
Reference	For l yeah the whistle So we can if financially voice recognition is not feasible we could go for a whistle

Failure: correct reply, wrong correction

Decode	Right **[attributes you've got lots of nom]** something and then night something and then end something
Request	I didn't catch what you said between 'Right' and 'something and'. Could you repeat it?
Reply	Yeah, you've got lots of Nom
Corrected	Right you've got lots of nom something and then night something and then end something
Reference	Oh yeah You've got lots of nom something and then NITE something and then N_ something

Failure: correct reply, wrong correction

Decode	**[Very so we've]** got twenty cents for our docking station.
Request	What did you say before 'got twenty'?
Reply	Very subtitles buttons yeah seven yeah so we've
Corrected	So we've got twenty cents for our docking station.
Reference	Very Subtitles buttons Yeah Seven Yeah So we've got twenty cents for our docking station huh

J Out-of-Domain Generalization with Matched Training Size

The source corpora of Table 2 have far more training data than their targets (62.9 vs. 6.2 hours and 28.5 vs. 0.7 hours; Table 5). To separate the effect of the corpus from that of the training size, we train a CF-Gain Scorer on a subset of the source's training set with as many hours as the target's training set. The subset is built from whole sessions: the sessions are visited in a fixed random order and their utterances are added until the target duration is reached, so the subset holds a few complete sessions and part of one, like a smaller corpus rather than a random sample of turns. This gives 11 complete AMI meetings and part of a twelfth, out of 134 training meetings, and part of one of the 16 CHiME-6 training sessions. The scorer is otherwise trained as in §4.1, with model selection and calibration on the source's development set, and applied unchanged to the target.

Table 6 shows the results. With matched training size, the transferred scorer falls below the in-domain scorer on both pairs: on NOTSOFAR-1 by 4.0 points of AP and 11.1 points of recall at 70% precision, and on DiPCo by 2.3 points of AP and 6.7 points of recall. The gains of Table 2 therefore come from the larger training set of the source corpus: at equal size, in-domain data is worth more, but a larger corpus of the same genre can replace it. The CHiME-6 subset comes from a single session, so its result likely depends on which session is drawn.

Test	Trained on	Hours	AP	Recall at prec. 50	Recall at prec. 70
NOTSOFAR-1	Entropy	–	39.2	28.6	6.3
	NOTSOFAR-1	6.2	59.8	68.7	39.4
	AMI	62.9	**63.2**	**71.7**	**39.7**
	AMI, subset	6.2	55.8	60.9	28.3
DiPCo	Entropy	–	41.3	29.0	11.0
	DiPCo	0.7	44.1	36.6	10.0
	CHiME-6	28.5	**54.8**	**58.0**	**24.4**
	CHiME-6, subset	0.7	41.8	31.4	3.3

Table 6: Out-of-domain generalization with matched training size. A CF-Gain Scorer trained on the other corpus of the same genre, on all of its training set or on a subset with as many hours as the target's training set, is applied unchanged to the target and compared with a scorer trained on the target and with the training-free entropy baseline. All numbers except hours are in percent. Best result per block in bold.

K Speech Enhancement Baseline

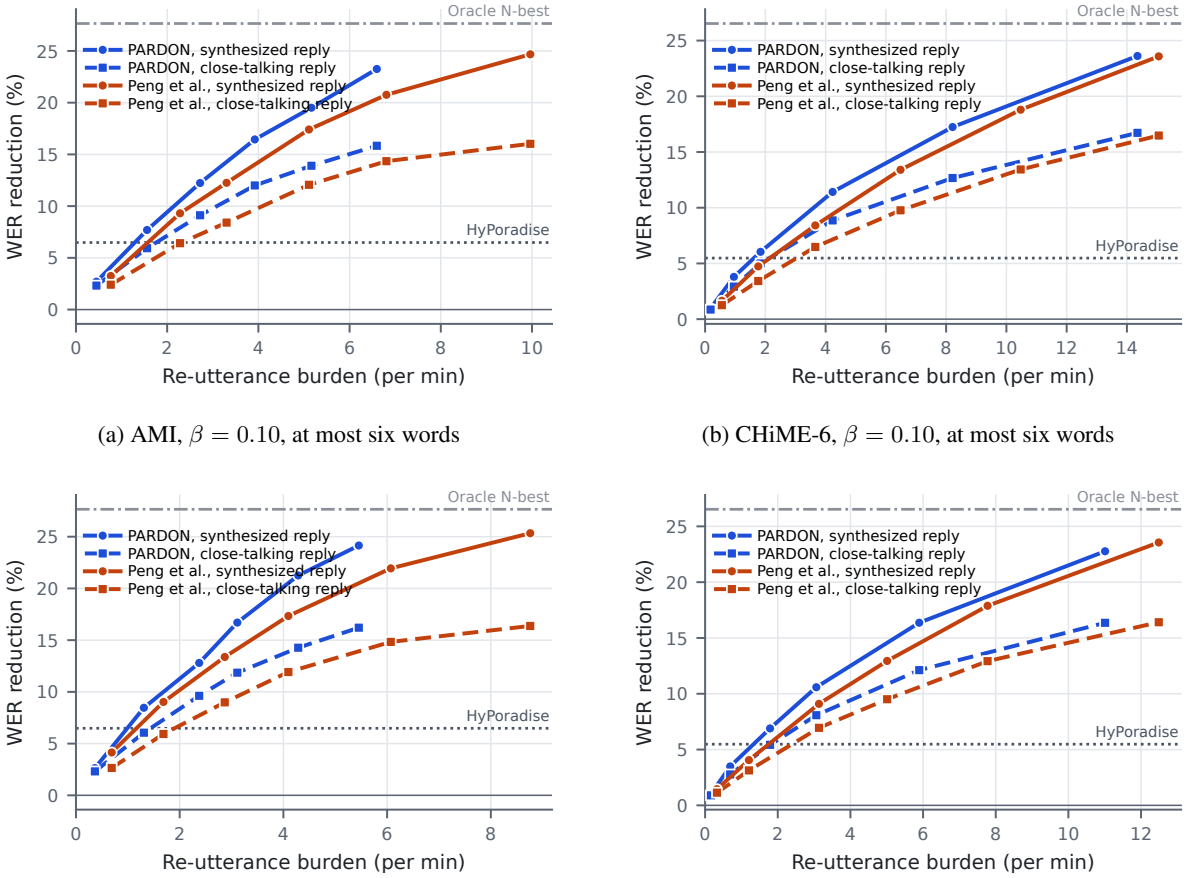
The far-field recording is passed through DeepFilterNet3 (Schröter et al., 2023), a causal noise-suppression model with about 40 ms of algorithmic latency that suits a streaming recognizer, and the enhanced audio is decoded by the same recognizer. The model runs at 48 kHz; the 16 kHz input is resampled up and the output resampled back. We cap its attenuation at a limit fitted on the development set. Enhancement increases the WER on both evaluation sets, from 30.0% to 32.4% on AMI (relative WER reduction -8.0%) and from 64.1% to 67.0% on CHiME-6 (-4.5%), so it is not shown in Figure 4.

L Additional Recourse Results

Figure 7 repeats the recourse comparison of §5.2 with a higher weight on re-uttered words ($\beta = 0.10$, spans of at most six words) and with longer spans (at most eight words, $\beta = 0.05$). Operating points are selected on the development set as in §M.

M Tuning Cost Parameters

We find operating points on the development set by minimizing the burden for different budgets of recall, the share of erroneous words, counting



(a) AMI, $\beta = 0.10$, at most six words

(b) CHiME-6, $\beta = 0.10$, at most six words

(c) AMI, $\beta = 0.05$, at most eight words

(d) CHiME-6, $\beta = 0.05$, at most eight words

Figure 7: Recourse on the AMI and CHiME-6 evaluation sets with a higher weight on re-uttered words (a, b) and with longer spans (c, d): relative WER reduction against the re-utterance burden, at the operating points selected on the development set, with synthesized or close-talking replies. Horizontal lines: the zero-interruption N-best correction of HyPoradise and the N-best oracle.

each deleted word, that fall inside a span or in the gap before one of its words. Recall approximates the ceiling on the error reduction our system can achieve. It is measured against the close-talking decode, so no gold transcript is used: a substituted or inserted word counts when it lies inside a span, and the deleted words of a gap count when the gap lies inside a span or at its boundary. Utterances with an empty noisy decode stay in the denominator.

Operating points are chosen separately for each scorer, span cap W and weight β. We sweep $c_R \in \{0.3, 0.4, \ldots, 0.8\}$ and 64 values of c_I spaced geometrically from 0.02 to 20. The recall budgets are six values spaced evenly from 5% to 90% of the lower of the two scorers' highest development recall, so that both scorers can reach every budget. For each budget, the pair (c_R, c_I) with the lowest development burden among those that reach it is applied unchanged to the evaluation set. The zero-interruption baselines are reported as single points.

N Training Details

Supervision. The close-talking recording is transcribed by the same ASR system as the far-field audio, and this transcript serves as the clean decode for supervision. Both noisy and clean decodes are normalized to words (lower-cased, with punctuation removed) before a pair HMM aligns them. Word-level substitution and insertion labels are copied to every subword token of the noisy word, and deletion labels, together with $\tilde{d}_j$ and $\tilde{\ell}_j$, only to its first token. The pair HMM is initialized from the hard-alignment counts, fit with ten EM iterations on the training utterances, and frozen before the CF-Gain Scorer is trained.

Features. Encoder states are pooled from the 13th and the final (24th) encoder blocks under the emission and dwell posteriors, giving four 1024-

dimensional vectors per token, and are concatenated with the predictor state and the scalar features of §3.2.1. Voice activity probabilities come from Silero VAD (Silero Team, 2021).

Training. The CF-Gain Scorer is a single-layer unidirectional LSTM with 128 hidden units, followed by three heads, each a feed-forward network with one hidden layer of 64 units. It is trained with AdamW (Loshchilov and Hutter, 2019) at a learning rate of 10^{-3} and weight decay 10^{-4}, on batches of 64 utterances, for up to 60 epochs with early stopping on development-set AP of e_j (patience 15). We then apply Platt scaling to e_j on the development set, and to d_j on its error tokens. A separate scorer is trained on each corpus.

Compute. All experiments run on a single NVIDIA L4 GPU. On AMI, the largest corpus, extracting the lattice features of the training set takes about 4 hours and training the CF-Gain Scorer about 30 minutes; all experiments in this paper, including the baselines, ablations, speech synthesis and recourse evaluation, took about 40 GPU hours in total. The LLM corrector and the N-best baseline are queried through the Gemini API. All results are from a single training run per setting.